Lombard SUI

Security Assessment

February 18th, 2025 — Prepared by OtterSec

Tuyết Dương tuyet@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-LSI-ADV-00 | Preventing Minting via Front-Running Payload 6

OS-LSI-ADV-01 | Denial of Service via Mint Limit Exhaustion 7

OS-LSI-ADV-02 | Missing Validator Set Integrity Checks 8

General Findings **9**

OS-LSI-SUG-00 | Prevention of Cross-Chain Replay Attacks 10

Appendices

Vulnerability Rating Scale **11**

Procedure **12**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Lombard Finance engaged OtterSec to assess the `bascule`, `consortium`, and `lbtc` programs. This assessment was conducted between January 30th and February 11th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 4 findings throughout this audit engagement.

In particular, we identified a vulnerability in which the functionality responsible for validating and storing the payload is susceptible to front-running. This allows an attacker to preemptively mark a valid payload as utilized, thereby preventing legitimate minting (OS-LSI-ADV-00). Additionally, during the configuration of the validator set, duplicate validators are not identified, and the correct key lengths are not enforced. Furthermore, there is no assertion to ensure that the threshold does not exceed the total weight (OS-LSI-ADV-02).

Furthermore, an attacker can repeatedly claim and redeem small coin amounts to exhaust the minting limit for the epoch, preventing other users from minting new tokens. This effectively creates a denial-of-service scenario (OS-LSI-ADV-01).

Additionally, we recommended modifying the fee payload structure to include the chain ID to prevent potential replay attacks (OS-LSI-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/lombard-finance/sui-contracts. This audit was performed against b3e614c.

A brief description of the programs is as follows:

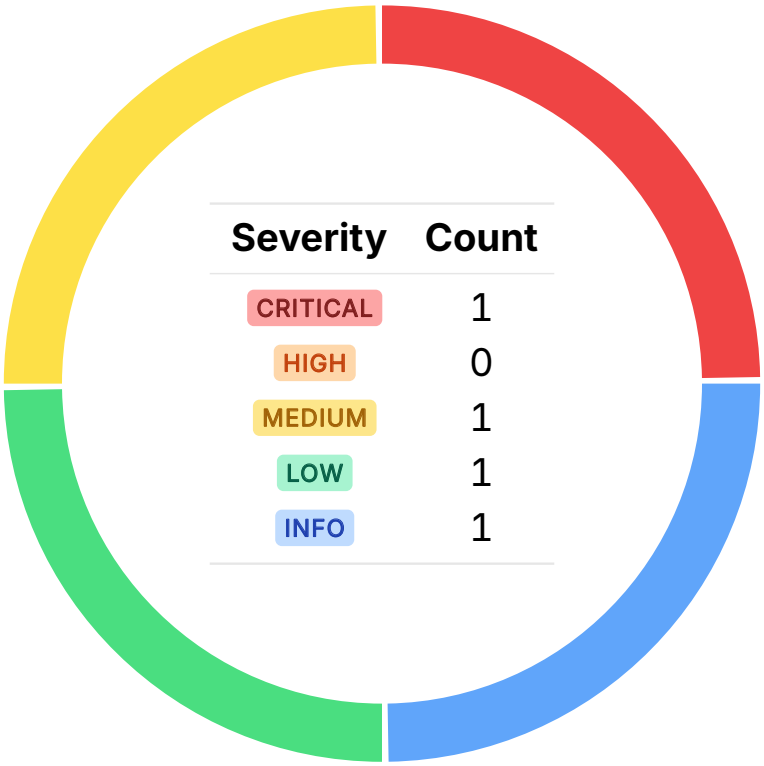
Name	Description
bascule	A cross-chain bridge contract that manages deposits and withdrawals with validation logic. It tracks deposits made on a source chain and allows corresponding withdrawals on the destination chain, ensuring security through validation thresholds.
Consortium	It manages a decentralized validator set and its governance, enabling admins to initialize and update validator sets across epochs. It validates changes through secure signature checks, payload validation, and compliance with weight thresholds.
lbtc	It creates the LBTC coin (a regulated asset-backed 1:1 by BTC) and sets up a treasury management system with a controlled treasury to manage the coin's inflow and outflow while ensuring secure coin metadata through freezing. A multisig address is used to control the treasury, adding an extra layer of security.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	1
HIGH	0
MEDIUM	1
LOW	1
INFO	1

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LSI-ADV-00	CRITICAL	RESOLVED	`validate_and_store_payload` is vulnerable to front-running, allowing an attacker to pre-emptively mark a valid payload as utilized, preventing legitimate minting.
OS-LSI-ADV-01	MEDIUM	RESOLVED	It is possible to repeatedly call `claim_native` and `return_native` with a small coin amount to exhaust the minting limit for the epoch, preventing other users from minting new tokens.
OS-LSI-ADV-02	LOW	RESOLVED	`assert_and_configure_validator_set` does not validate duplicate validators, enforce correct key lengths, or ensure that the threshold does not exceed the total weight.

© 2025 Otter Audits LLC. All Rights Reserved.

Preventing Minting via Front-Running Payload CRITICAL OS-LSI-ADV-00

Description

`validate_and_store_payload` in `Consortium` validates and records the usage of a payload. However, since it is public, anyone may call it. This creates a potential front-running attack, where a malicious actor may interfere with the expected execution of a valid transaction. Thus, if a user submits a valid transaction that calls `validate_and_store_payload(payload, proof)` to validate the payload, the attacker may front-run this with a similar call to `validate_and_store_payload`, which executes before the user's call.

>_ *move/consortium/sources/consortium.move* RUST

```rust
public fun validate_and_store_payload(
    consortium: &mut Consortium,
    payload: vector<u8>,
    proof: vector<u8>,
) {
    // Payload should consist of 164 bytes (160 for message and 4 for the action)
    assert!(payload.length() == 164, EInvalidPayloadLength);
    let hash = hash::sha2_256(payload);
    assert!(!consortium.is_payload_used(hash), EUsedPayload);
    // get the signatures from the proof
    let signatures = payload_decoder::decode_signatures(proof);
    // get the validator set for the current epoch
    let signers = consortium.get_validator_set(consortium.epoch);
    assert!(validate_signatures(signers.pub_keys, signatures, signers.weights,
        ↪  signers.weight_threshold, payload, hash), EInvalidPayload);
    consortium.used_payloads.add(hash, true);
}
```

This stores the payload's hash in `used_payloads`, marking it as used. Consequently, the user's call to `mint` will fail because `is_payload_used(hash)` will return true, preventing the minting of coins.

Remediation

Restrict `validate_and_store_payload` to be callable only by the `lbtc` package or ensure that payload tracking is managed within the treasury module.

Patch

Resolved in 0fd60cf.

© 2025 Otter Audits LLC. All Rights Reserved.

Denial of Service via Mint Limit Exhaustion MEDIUM OS-LSI-ADV-01

Description

`claim_native` and `return_native` in `bridge` allow users to swap wrapped tokens (`WT`) for native tokens and vice versa. However, an attacker can repeatedly cycle a small token amount between minting (`claim_native`) and burning (`return_native`). Since each minting operation contributes to the epoch's minting limit, this exploitation will exhaust the treasury's capacity within an epoch, preventing legitimate users from minting new tokens.

Remediation

Remove the ability for the user to wrap and unwrap purely inside sui and forces them to go cross-chain, which should significantly slow down such an attac.

Patch

Resolved in a58183d.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validator Set Integrity Checks LOW OS-LSI-ADV-02

Description

The current implementation of `assert_and_configure_validator_set` in `Consortium` lacks critical checks, which may allow invalid validator keys. The function does not check for duplicate validator public keys. There is no validation to ensure that the validator keys are correct and that each validator's public key is exactly 65 bytes long.

>_ *move/consortium/sources/consortium.move* RUST

```rust
fun assert_and_configure_validator_set(
    consortium: &mut Consortium,
    action: u32,
    validators: vector<vector<u8>>,
    weights: vector<u256>,
    weight_threshold: u256,
    epoch: u256,
) {
    assert!(action == consortium.valset_action, EInvalidAction);
    assert!(validators.length() >= MIN_VALIDATOR_SET_SIZE, EInvalidValidatorSetSize);
    assert!(validators.length() <= MAX_VALIDATOR_SET_SIZE, EInvalidValidatorSetSize);
    assert!(weight_threshold > 0, EInvalidThreshold);
    assert!(validators.length() == weights.length(), EInvalidValidatorsAndWeights);
    [...]
}
```

Remediation

Ensure that each key in the validators list is exactly 65 bytes long.

Patch

Resolved in f825f07.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LSI-SUG-00	Adding `chain_id` to the fee payload, similar to the mint payload, prevents replay attacks.

© 2025 Otter Audits LLC. All Rights Reserved.

Prevention of Cross-Chain Replay Attacks OS-LSI-SUG-00

Description

Add `chain_id` to the fee payload, similar to how it is included in the mint payload. This prevents replay attacks, where a transaction could be relayed on a different chain. Adding `chain_id` ensures that signed fee transactions are only valid on the intended chain, preventing cross-chain replay attacks.

Remediation

Modify the fee payload structure to include `chain_id`.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.